Проектирование архитектуры и конструирование ПО*

Проектирование ОС, Технологии, Инструменты

Лекция 3

Технология АОМК

Интерфейс IEcoUnknown



Интерфейс IEcoUnknown

- *Неизвестный* интерфейс, о котором знают **ВСЕ**, кто использует ***АОМК технологию***!

Mr. Unknown

- Интерфейс IEcoUnknown:
  - метод QueryInterface - запрос интерфейса
  - методы AddRef и Release - подсчет ссылок



What's your name?





Определение интерфейса IEcoUnknown на Си

```
/* IEcoUnknown IID = 00000000-0000-0000-0000-0000000000AA */
#ifndef __IID_IEcoUnknown
static const UGUID IID_IEcoUnknown = { 0x01, 0x10, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
                                       0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0xAA };
#endif /* __IID_IEcoUnknown */

typedef struct IEcoUnknown* IEcoUnknownPtr_t;

typedef struct IEcoUnknownVTbl {

    /* IEcoUnknown */
    int16_t (*QueryInterface)(/* in */ IEcoUnknownPtr_t me, /* in */ const UGUID* riid,
                              /* out */ void** ppv);
    uint32_t (*AddRef)(/* in */ IEcoUnknownPtr_t me);
    uint32_t (*Release)(/* in */ IEcoUnknownPtr_t me);

} IEcoUnknownVTbl;

interface IEcoUnknown {
    struct IEcoUnknownVTbl *pVTbl;
} IEcoUnknown;
```



Правильный интерфейс

- Все интерфейсы АОМК наследуют IEcoUnknown, в каждом интерфейсе есть функции QueryInterface, AddRef, Release:

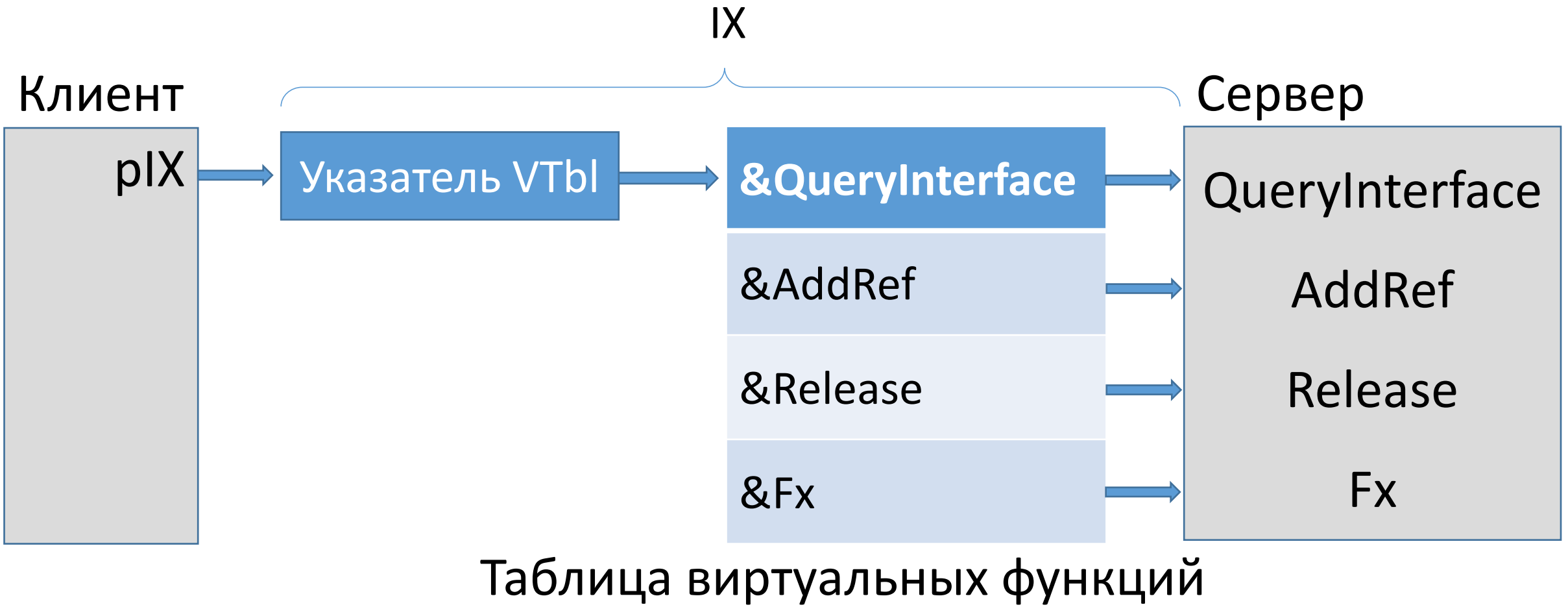


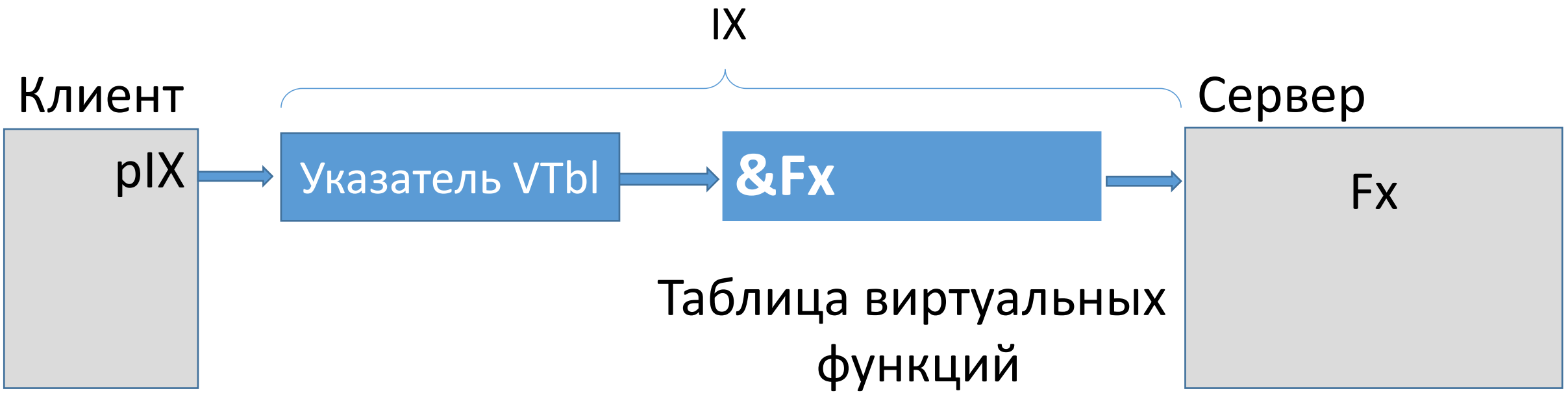
Таблица виртуальных функций



НЕправильный интерфейс



- Если интерфейс не наследует IEcoUnknown, т.е. в интерфейсе отсутствуют функции QueryInterface, AddRef, Release, то это не интерфейс АОМК:





Функция QueryInterface

- ***IEcoUnknown*** содержит функцию-член ***QueryInterface***, при помощи которой клиент определяет, поддерживается ли тот или иной интерфейс. ***QueryInterface*** возвращает указатель на интерфейс, если компонент его поддерживает; в противном случае возвращается код ошибки (тогда клиент может запросить указатель на другой интерфейс или аккуратно выгрузить компонент).
- У ***QueryInterface*** два основных параметра:
  - IID – идентификатор интерфейса
  - Адрес – указатель на интерфейс

```
int16_t (*QueryInterface)(/* in */ struct IEcoUnknown* me,
                          /* in */ const UGUID* riid, /* out */ void** ppv);
```

ПРИМЕЧАНИЕ: При описании интерфейса на СИ, первым параметром идет указатель ME (THIS) на экземпляр компонента.



Реализация QueryInterface

- Реализация QueryInterface сводиться к операции возврата указателя интерфейса, соответствующего данному IID (идентификатору интерфейса).

```
int16_t QueryInterface(/* in */ struct IX* me, /* in */ const UGUID* riid,
                       /* out */ void** ppv) {
    CX* pCMe = (CX*)me;
    if ( IsEqualUGUID(riid, &IID_IX) ) {
        *ppv = &pCMe->m_pVTblIX;
        pCMe->m_pVTblIX->AddRef((IX*)pCMe);
    }
    else if ( IsEqualUGUID(riid, &IID_IEcoUnknown) ) {
        *ppv = &pCMe->m_pVTblIX;
        pCMe->m_pVTblIX->AddRef((IX*)pCMe);
    }
    else {
       *ppv = 0;
       return -1;
    }
    return 0;
}
```



Правила и соглашения QueryInterface

- Вы всегда получаете один и тот же IEcoUnknown.
- Вы можете получить интерфейс снова, если смогли получить его раньше.
- Вы можете снова получить интерфейс, который у Вас уже есть.
- Вы всегда можете вернуться туда, откуда начали.
- Если Вы смогли попасть куда-то хоть откуда-нибудь, Вы можете попасть туда откуда угодно.





Вы всегда получаете один и тот же IEcoUnknown

- У данного экземпляра компонента есть только один интерфейс ***IEcoUnknown***.

- Всегда, когда Вы запрашиваете у компонента ***IEcoUnknown***, не важно через какой интерфейс, в ответ вы получите одно и тоже значение указателя.

- Вы можете определить, указывают ли два интерфейса на один компонент, запросив у каждого из них ***IEcoUnknown*** и сравнив результаты.



Вы всегда получаете один и тот же IEcoUnknown

Пример:

```
bool_t Same(/* in */ IX* pIX, /* in */ IY* pIY) {
    IEcoUnknown* pI1 = 0;
    IEcoUnknown* pI2 = 0;

    /* Получить указатель на IEcoUnknown через pIX */
    pIX->m_pVTbl->QueryInterface(pIX, &IID_IEcoUnknown, (void **)&pI1);
    /* Получить указатель на IEcoUnknown через pIY */
    pIY->m_pVTbl->QueryInterface(pIY, &IID_IEcoUnknown, (void **)&pI2);

    /* Сравнить полученные указатели */
    if (pI1 == pI2) {
        return 1;
    }
    return 0;
}
```



Вы можете получить интерфейс снова, если смогли получить его раньше.

- Если ***QueryInterface*** однажды успешно обработала запрос на некоторый интерфейс, то все последующие запросы для того же компонента будут успешными.

- Если же запрос был неудачным, то для этого интерфейса ***QueryInterface*** всегда будет возвращать ошибку. Это правило относится только к конкретному экземпляру компонента. К вновь созданному экземпляру оно неприменимо.



Вы можете получить интерфейс снова, если смогли получить его раньше. *Пример*:

```
bool_t Again(/* in */ IX* pIX) {
    IY* pIY = 0;
    IY* pIY2 = 0;

    /* Получить указатель на IY через IX */
    if (pIX->m_pVTbl->QueryInterface(pIX, &IID_IY, (void **)&pIY) != 0) {

        /* Получить указатель на IY через IX */
        if (pIX->m_pVTbl->QueryInterface(pIX, &IID_IY, (void **)&pIY2) != 0) {
            return 1;
        }
    }

    return 0;
}
```



Вы можете снова получить интерфейс, который у Вас уже есть.

- Если у Вас есть интерфейс IX, то Вы можете запросить через него интерфейс IX и получите в ответ указатель на IX.

- Это правило вытекает из того, что все интерфейсы полиморфны относительно ***IEcoUnknown*** и многим функциям передается указатель на ***IEcoUnknown***.

- У этих функций должна быть возможность использовать любой указатель на ***IEcoUnknown*** и получить по нему любой другой интерфейс.



Вы можете снова получить интерфейс, который у Вас уже есть. *Пример:*

```
IX* Exist(/* in */ IEcoUnknown* pIUnk) {
    IX* pIX = 0;
    IX* pIX2 = 0;

    /* Запросить IX у IEcoUnknown */
    if (pIUnk->m_pVTbl->QueryInterface(pIUnk,IID_IX,(void**)&pIX) != 0) {
        /* Запросить IX у IX */
        pIX->m_pVTbl->QueryInterface(pIX, IID_IX, (void**)&pIX2);
    }

    return pIX2;
}
```



Вы всегда можете вернуться туда, откуда начали.

- Если у Вас есть указатель на интерфейс IX и с его помощью Вы успешно получаете интерфейс IY, то можно получить «обратно» интерфейс IX через указатель на IY.

- Иными словами, независимо от того, какой интерфейс у Вас есть сейчас, можно снова получить интерфейс, с которого Вы начали.



Вы всегда можете вернуться туда, откуда начали.

Пример:

```
IX* Back(/* in */ IX* pIX) {
    IX* pIX2 = 0;
    IY* pIY = 0;

    /* Получить IY через IX */
    if (pIX->m_pVTbl->QueryInterface(pIX, IID_IY, (void**)&pIY) != 0) {

        /* Получить IX через IY */
        pIY->m_pVTbl->QueryInterface(pIY, IID_IX, (void**)&pIX2);
    }

    return pIX2;
}
```



Если Вы смогли попасть куда-то хоть откуда-нибудь, Вы можете попасть туда откуда угодно.

- Если Вы можете получить у компонента некоторый интерфейс, то его можно получить с помощью любого из интерфейсов, поддерживаемых компонентом.

- Если можно получить интерфейс IY через IX, а IZ — через IY, то IZ можно получить и через IX.



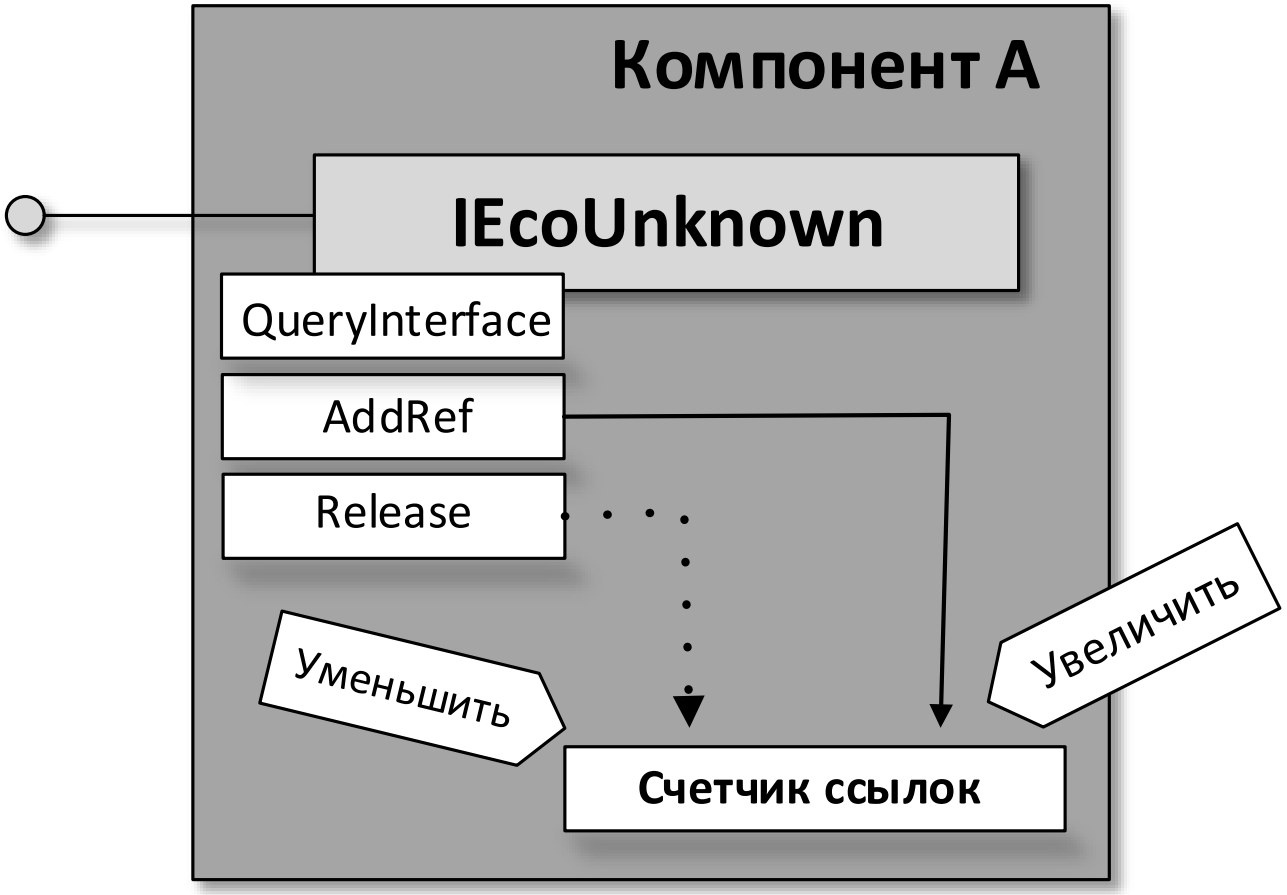
Если Вы смогли попасть куда-то хоть откуда-нибудь, Вы можете попасть туда откуда угодно.

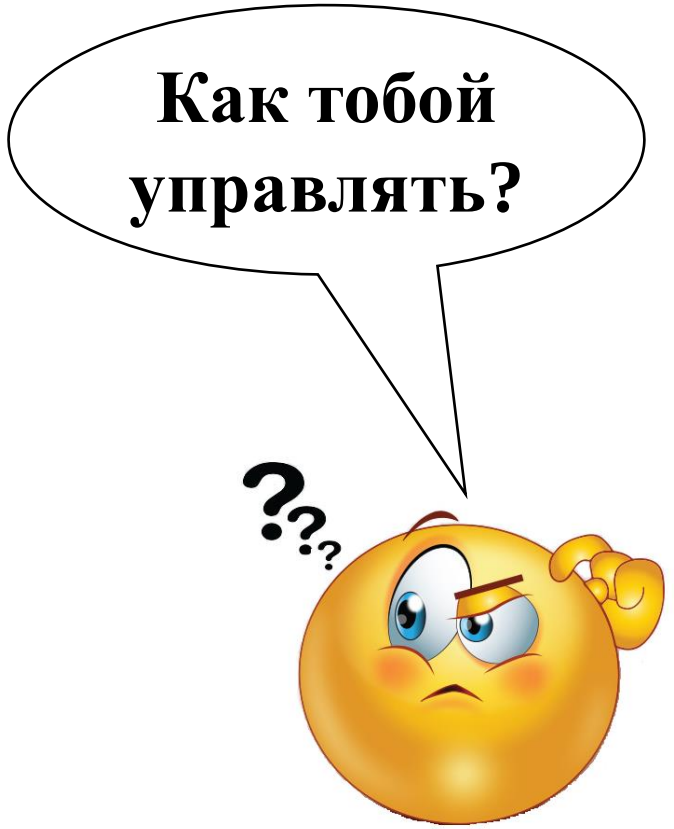
```
int16_t Any(/* in */ IX* pIX) {
    IY* pIY = 0;
    IZ* pIZ = 0;

    /* Запросить IY у IX */
    if (pIX->m_pVTbl->QueryInterface(pIX, IID_IY, (void**)&pIY) != 0) {
        /* Запросить IZ и IY */
        if (pIY->m_pVTbl->QueryInterface(pIY, IID_IZ, (void**)&pIZ) != 0) {
            /* Запросить IZ у IX */
            if (pIX->m_pVTbl->QueryInterface(pIX, IID_IZ, (void**)&pIZ) != 0) {
                return 0;
            }
        }
    }
    return -1;
}
```

Подсчет ссылок

- ***AddRef*** и ***Release*** реализуют технику управления памятью, известную как ***подсчет ссылок*** (*reference counting*).







Подсчет ссылок

- Когда клиент получает некоторый интерфейс, значение этого счетчика увеличивается на единицу.



- Когда клиент заканчивает работу с интерфейсом, значение на единицу уменьшается.



- Когда оно доходит до нуля, компонент удаляет себя из памяти.

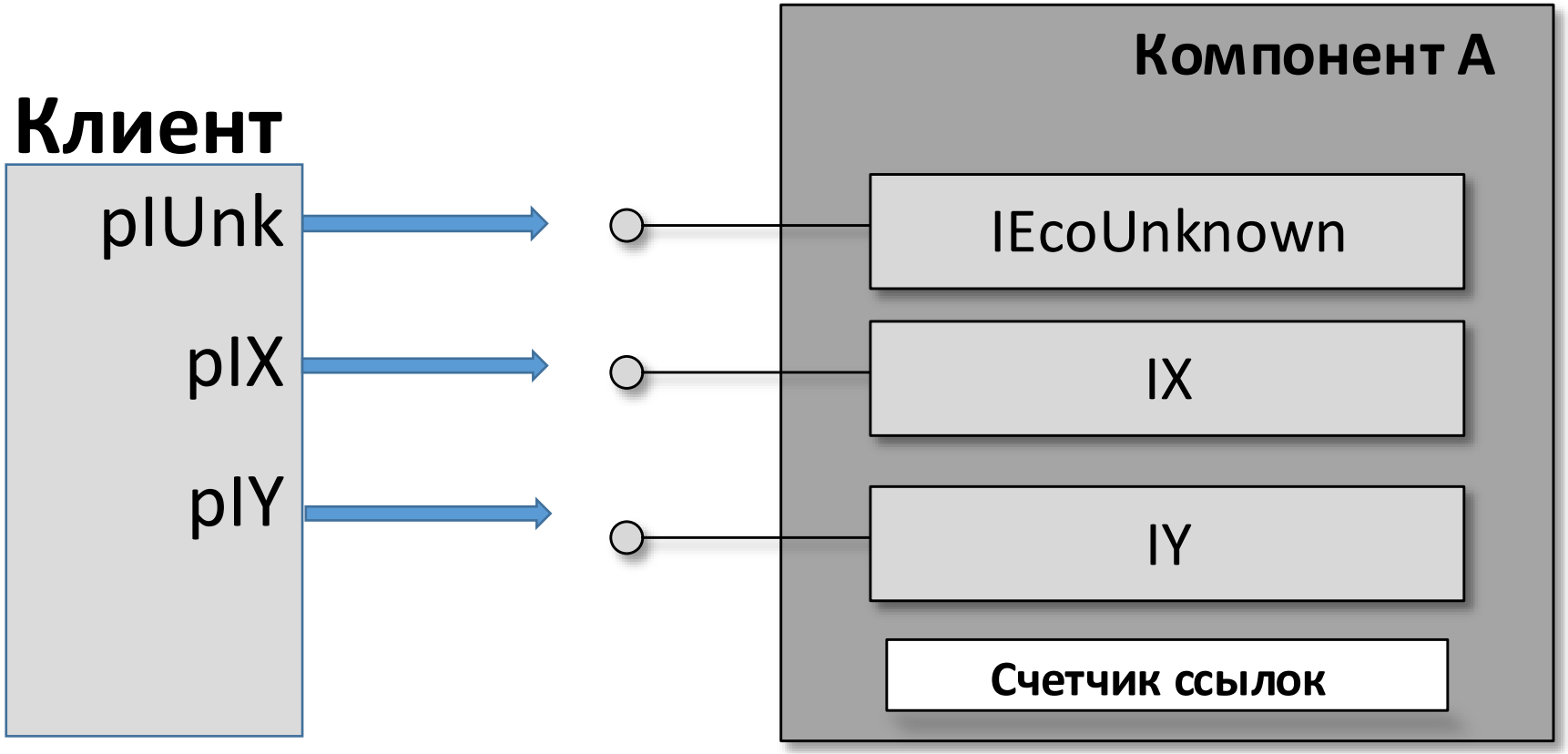


- Клиент также увеличивает счетчик ссылок, когда создает новую ссылку на уже имеющийся у него интерфейс.



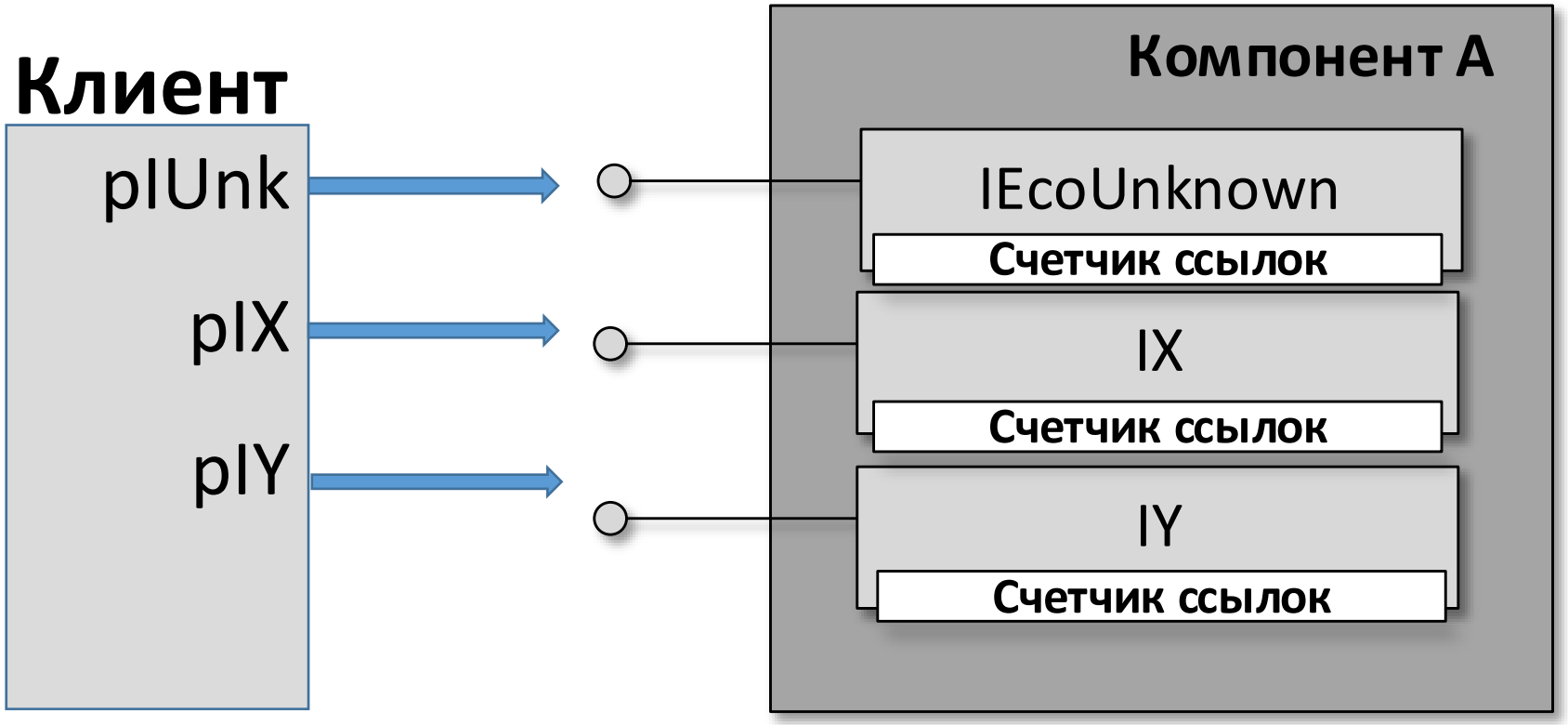
Счетчики ссылок

- Общий счетчик ссылок.



Счетчики ссылок

- Отдельные счетчики ссылок для интерфейсов.
  - для упрощения отладки
  - для поддержки выделения ресурсов по требованию





Реализация AddRef и Release

- Реализация AddRef сводиться к операции увеличения на единицу.

```
uint32_t AddRef(/* in */ struct IX* me) {
    CX* pCMe = (CX*)me;

    /* Проверка указателя */
    if (me == 0 ) {
        return -1;
    }

    /* Увеличение счетчика ссылок на компонент */
    return ++pCMe->m_cRef;
}
```



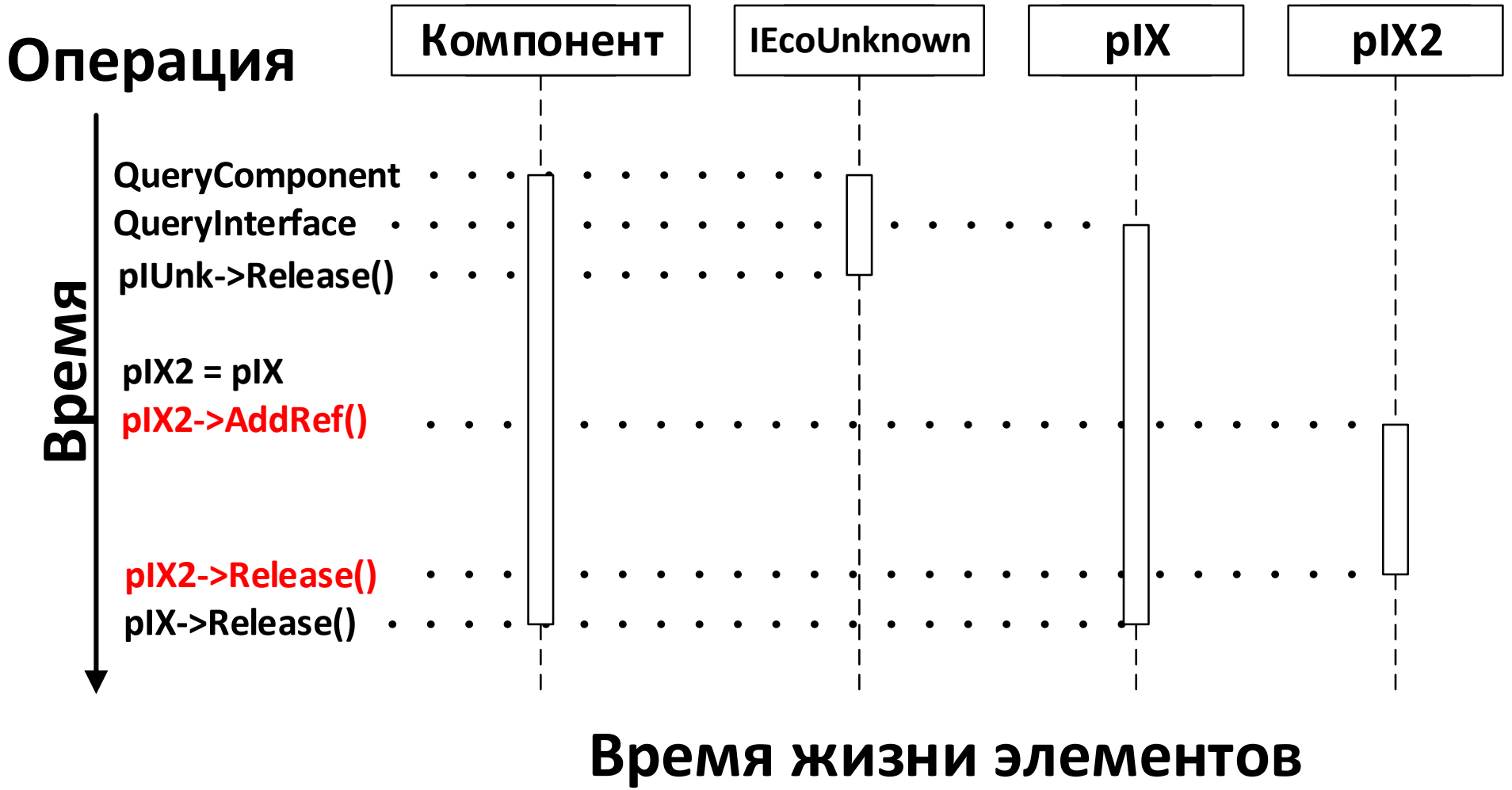
Реализация AddRef и Release

- Реализация Release сводиться к операции уменьшения на единицу.

```
uint32_t Release(/* in */ struct IX* me) {
    CX* pCMe = (CX*)me;
    /* Проверка указателя */
    if (me == 0 ) {
        return -1;
    }
    /* Уменьшение счетчика ссылок на компонент */
    --pCMe->m_cRef;
    /* В случае обнуления счетчика, освобождение данных экземпляра */
    if ( pCMe->m_cRef == 0 ) {
        deleteCX(me);
        return 0;
    }
    return pCMe->m_cRef;
}
```

Вложенность времен жизни указателей на интерфейсы.

- Ссылки для указателя со вложенным временем жизни подсчитывать не требуется.





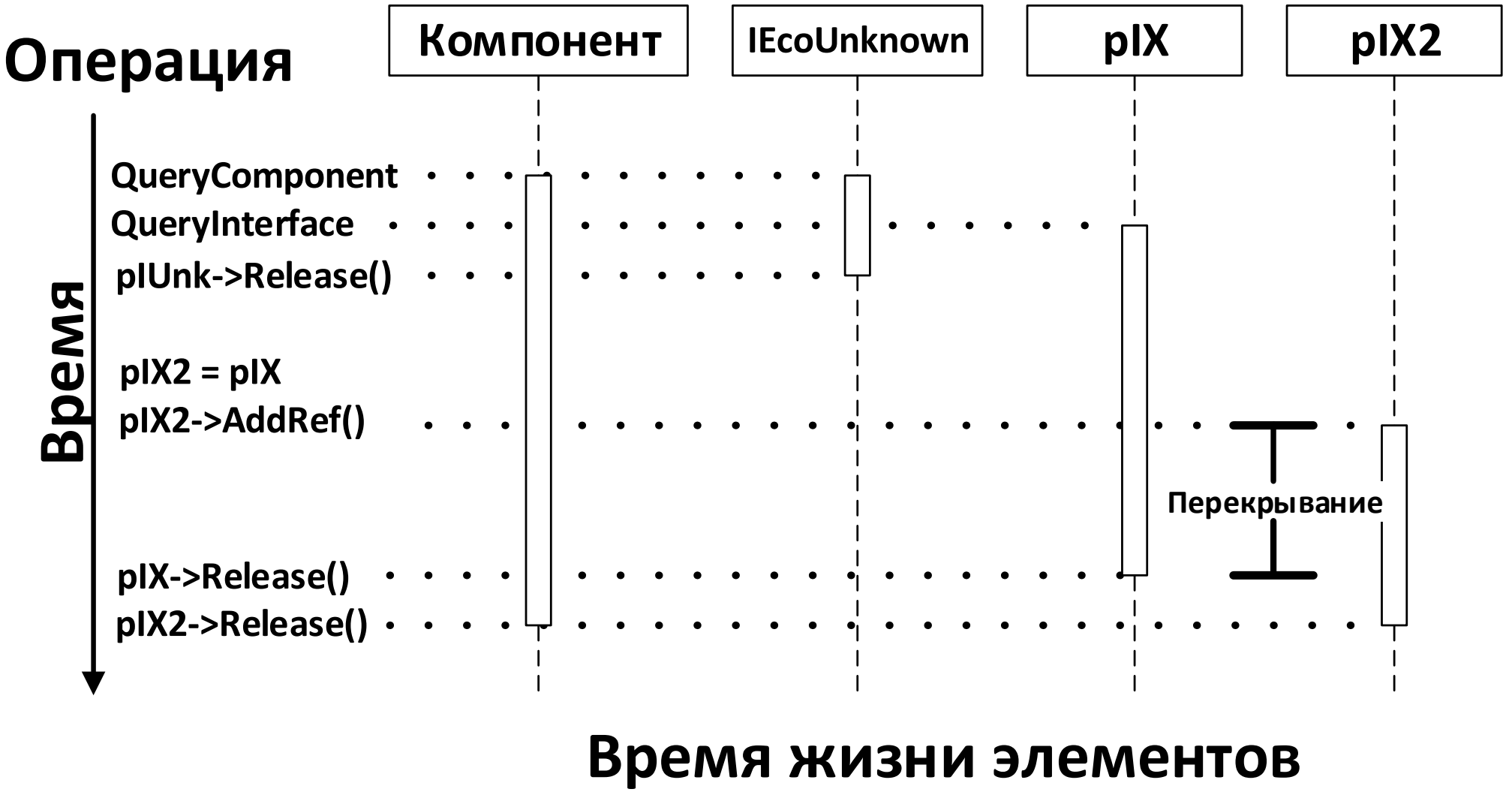
Вложенность времен жизни указателей на интерфейсы.

- ***Пример:***

```
/* Получение интерфейса */
pIBus->pVTbl->QueryComponent(pIBus, &CID_IX, 0, &IID_IX, (void**) &pIX);
/* Скопировать pIX */
pIX2 = pIX;
/* Начало жизни pIX2 - увеличить счетчик */
pIX2->pVTbl->AddRef(pIX2);
/* Использовать интерфейс IX */
pIX->pVTbl->Fx(pIX);
pIX2->pVTbl->Fx(pIX2);
/* Завершить работу с pIX2 – уменьшить счетчик */
pIX2->pVTbl->Release(pIX2);
/* Завершить работу с pIX */
pIX->pVTbl->Release(pIX);
```

Перекрывание времен жизни указателей на интерфейсы.

- Необходимо подсчитывать ссылки на оба интерфейса.





Перекрывание времен жизни указателей на интерфейсы.

- ***Пример:***

```
/* Получение интерфейса */
pIBus->pVTbl->QueryComponent(pIBus, &CID_IX, 0, &IID_IX, (void**) &pIX);
/* Скопировать pIX */
pIX2 = pIX;
/* Начало жизни pIX2 - увеличить счетчик */
pIX2->pVTbl->AddRef(pIX2);
/* Использовать интерфейс IX */
pIX->pVTbl->Fx(pIX);
pIX2->pVTbl->Fx(pIX2);
/* Завершить работу с pIX – уменьшить счетчик */
pIX->pVTbl->Release(pIX);
/* Завершить работу с pIX2 */
pIX2->pVTbl->Release(pIX2);
```



Правила подсчета ссылок

- **Вызывайте AddRef перед возвратом.** Функции, возвращающие интерфейсы, перед возвратом всегда должны вызывать AddRef для соответствующего указателя. Это также относится к QueryInterface и функции QueryComponent. Таким образом, Вам не нужно вызывать AddRef в своей программе после получения (от функции) указателя на интерфейс.
- **По завершении работы вызывайте Release.** Когда Вы закончили работу с интерфейсом, следует вызвать для него Release.
- **Вызывайте AddRef после присваивания.** Когда бы Вы ни присваивали один указатель на интерфейс другому, вызывайте AddRef. Иными словами: следует увеличить счетчик ссылок каждый раз, когда создается новая ссылка на данный интерфейс.





Правило для выходных параметров.

- *Выходной параметр* (*out parameter*) — это параметр функции, в котором вызывающей программе возвращается некоторое значение.

- Любая функция, возвращающая указатель на интерфейс через выходной параметр или как свое собственное возвращаемое значение, должна вызывать *AddRef* для этого указателя.

- Это то же самое правило, что и **«Вызывайте *AddRef* перед возвратом»**



Правило для выходных параметров.

- Пример выходного параметра:

```
int16_t QueryInterface(/* in */ struct IX* me, /* in */ const UGUID* riid, /* out */ void** ppv) {
    CX* pCMe = (CX*)me;
    int16_t result = -1;
    /* Проверка и получение запрошенного интерфейса */
    if ( IsEqualUGUID(riid, &IID_IX) ) {
        *ppv = &pCMe->m_pVTblIX;
        pCMe->m_pVTblIX->AddRef((IX*)pCMe);
    }
    else if ( IsEqualUGUID(riid, &IID_IEcoUnknown) ) {
        *ppv = &pCMe->m_pVTblIX;
        pCMe->m_pVTblIX->AddRef((IX*)pCMe);
    }
    else {
        *ppv = 0;
        return -1;
    }
    return 0;
}
```



Правило для входных параметров.

- *Входной параметр* (*in parameter*) — это параметр, через который функции передается некоторое значение. Функция использует это значение, но не изменяет его и ничего не возвращает в нем вызывающей программе.

- Указатель на интерфейс, переданный в функцию, не требует обращений к *AddRef* и *Release*, так как время жизни функции всегда вложено во время жизни вызывающей программы.



Правило для входных параметров.

- ***Пример:***

```
void Foo(/* in */ struct IX* pIX) {
    /* Использование функции интерфейса */
    pIX->pVTbl->Fx(pIX);
}

. . .

IX* pIX = 0;

/* Получение интерфейса IX */
pIBus->pVTbl->QueryComponent(pIBus, &CID_IX, 0, &IID_IX, (void**) &pIX);
/* Вызов функции */
Foo(pIX);
/* Завершить работу с pIX */
pIX->pVTbl->Release(pIX);
```



Правило для параметров типа вход-выход.

- *Параметр типа вход-выход* (*in-out parameter*) может одновременно быть и входным, и выходным. Функция использует переданное ей значение такого параметра, затем изменяет его и возвращает вызывающей программе.

- Функция обязана вызвать *Release* для указателя на интерфейс, переданного ей как произвольный параметр, прежде чем записать на его место новый указатель. Перед возвратом в вызывающую программу функция также должна вызвать *AddRef* для нового значения параметра.



Правило для параметров типа вход-выход.

- ***Пример:***

```
void Exchange(/* in */ int16_t indx, /* in | out */ struct IX** ppIX) {
    /* Использование функции интерфейса */
    (*ppIX)->pVTbl->Fx((*ppIX));
    /* Завершить работу с ppIX */
    (*ppIX)->pVTbl->Release((*ppIX));

    /* Выбираем указатель интерфейса из кэша */
    *ppIX = g_Chache[indx];
    /* Увеличиваем счетчик для ppIX */
    (*ppIX)->pVTbl->AddRef((*ppIX));
}
```



Правило для локальных переменных.

- Локальные копии указателей на интерфейсы, конечно, существуют только во время выполнения функций и не требуют пар *AddRef* / *Release*. Это правило непосредственно вытекает из правила для входных параметров.

- Существование локального указателя на интерфейс pIX2, вложено во время жизни указателя pIX, переданного как входной параметр, - так что вызывать *AddRef* или *Release* для pIX2 не нужно.. .



Правило для локальных переменных.

- ***Пример:***

```
void Foo(/* in */ struct IX* pIX) {

        IX* pIX2 = 0;

        /* Скопировать pIX */
        pIX2 = pIX;

        /* Использование функции интерфейса */
        pIX2->pVTbl->Fx(pIX2);
    }
```



Правило для глобальных переменных.

- Если указатель на интерфейс сохраняется в глобальной переменной, то прежде чем передавать управление другой функции, необходимо вызвать *AddRef*. Поскольку переменная является глобальной, любая функция может вызвать *Release* и закончить жизнь этого указателя.

- Указатели на интерфейсы, сохраняемые в переменных членах, должны обрабатываться аналогично. Любая функция-член класса может изменить состояние такого указателя.



Правило для глобальных переменных.

- ***Пример:***

```
/* Объявление глобального указателя на интерфейс IX */
IX* g_pIX = 0;

. . .

void Foo(/* in */ struct IX* pIX) {

    /* Скопировать pIX */
    g_pIX = pIX;

    /* Увеличиваем счетчик для g_pIX */
    g_pIX->pVTbl->AddRef(g_pIX);
}
```



Правило для сомнительных случаев.

- Всякий раз, когда у Вас возникает сомнение, вставляйте пару *AddRef* / *Release*.

- Её отсутствие редко дает значительный выигрыш в производительности или экономию памяти, зато легко может привести к созданию компонента, который никогда не удаляется из памяти.

- Кроме того, Вы можете потратить много времени на поиск ошибки, вызванной неправильным подсчетом ссылок.



Правило для сомнительных случаев.

- ***Пример:***

```
void Foo(/* in */ struct IX* pIX) {

    /* Увеличиваем счетчик для pIX */
    pIX->pVTbl->AddRef(pIX);

    /* Использование функции интерфейса */
    pIX->pVTbl->Fx(pIX);

    /* Уменьшить счетчик для pIX */
    pIX->pVTbl->Release(pIX);
}
```

Спасибо за внимание!